

10. Idempotency

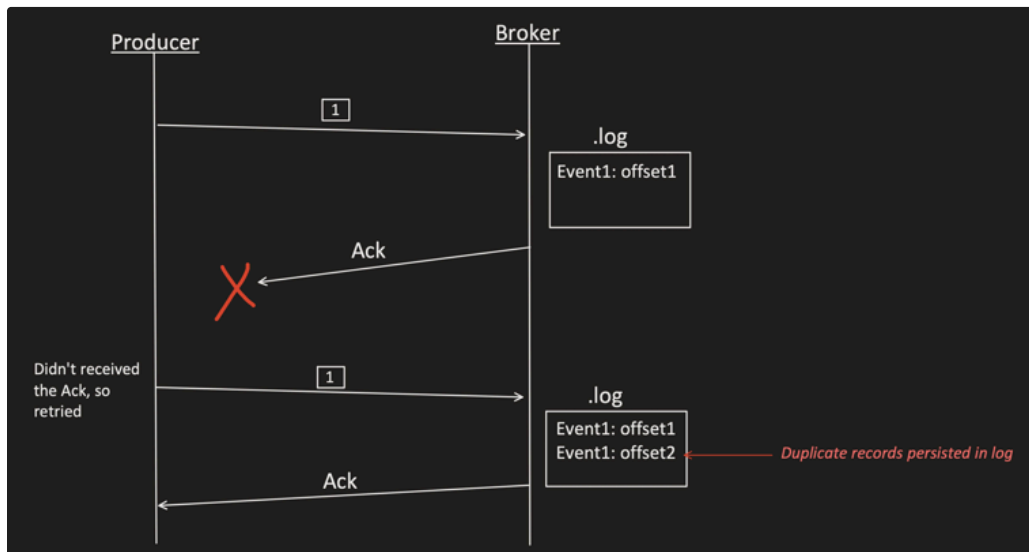
Lets understand the problem when producer set below configurations:

`max.in.flight.requests.per.connection > 1`

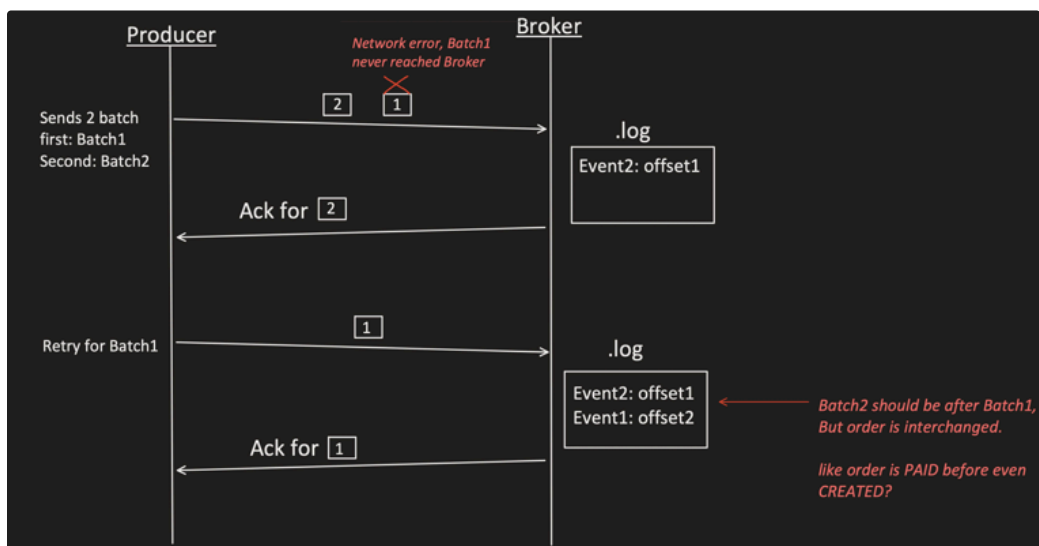
and

Retries is also `> 0`

Problem 1: Duplicate Messages



Problem 2: Message Reordering



Solution: set Idempotency = true

Producer: application.properties

```
1 | spring.kafka.producer.properties.enable.idempotence=true
2 |
```

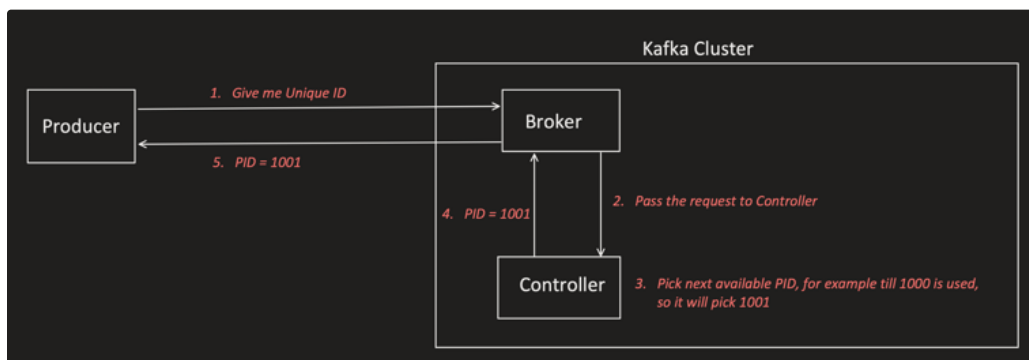
Note:

Kafka version < 3.0, default value of this property is FALSE

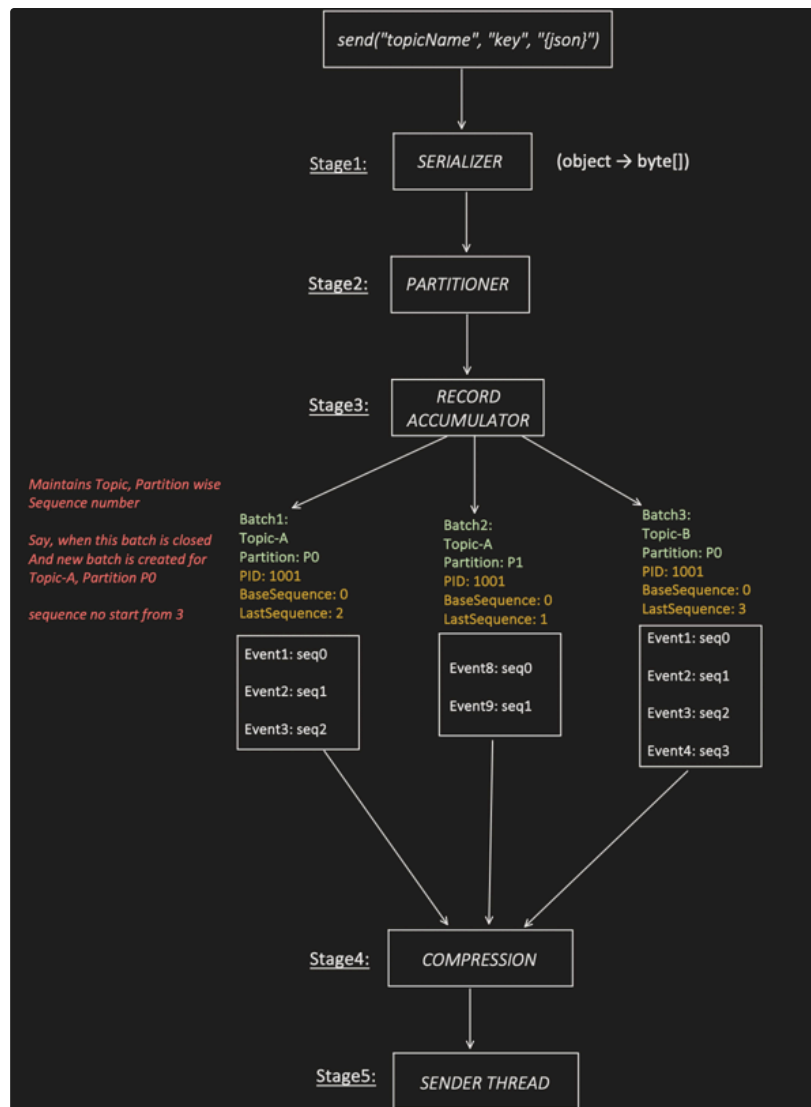
Kafka version >= 3.0, default value of this property is TRUE

How it works?

Step1: Producer application startup

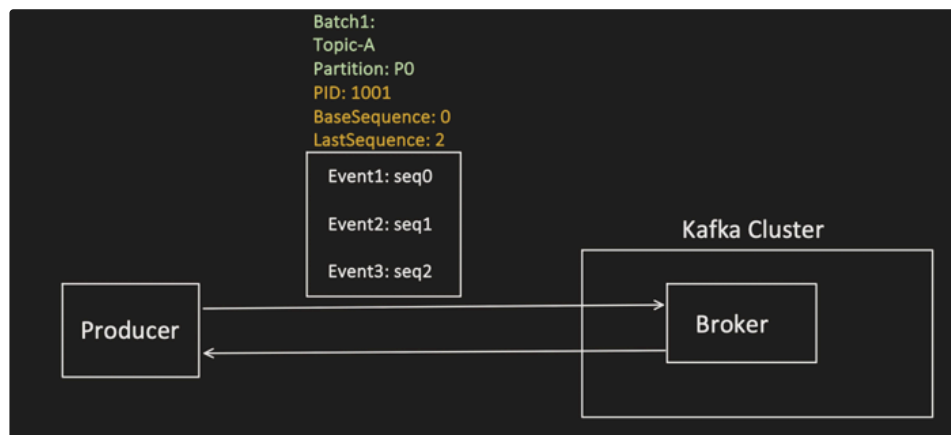


Step2: Producer send requests

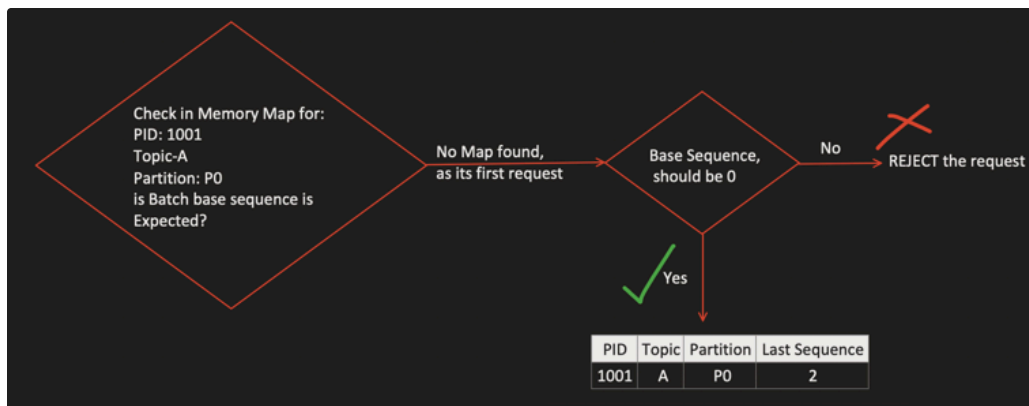


Step3: Batch received by Broker

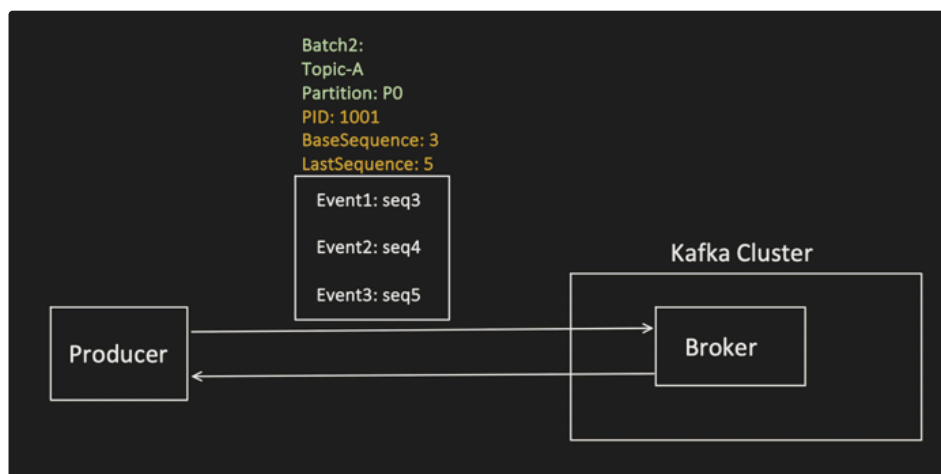
First request from Producer to Broker



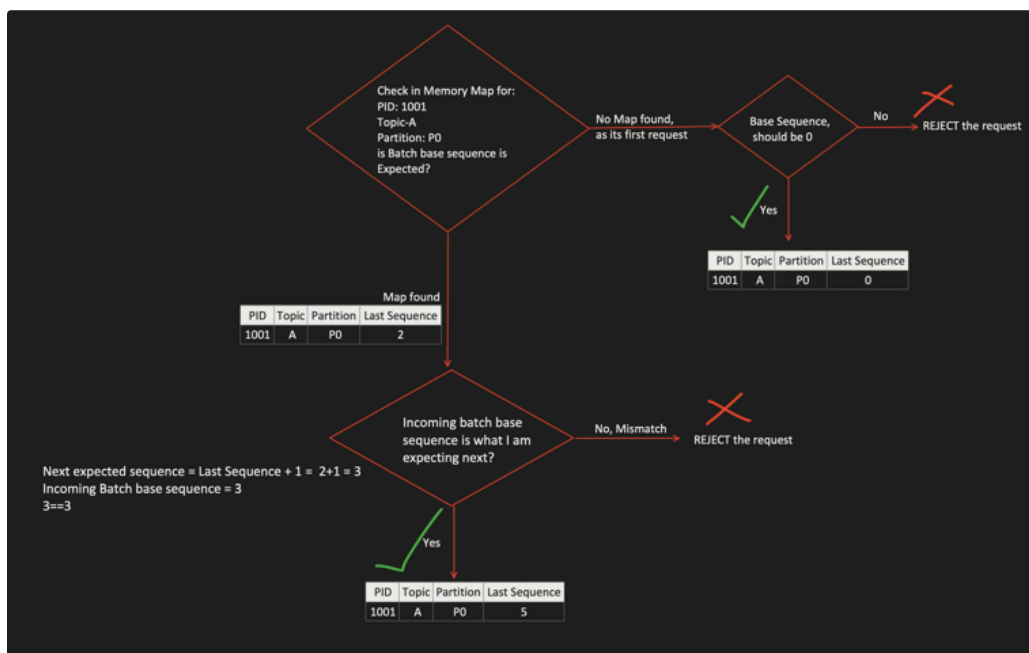
Now within Broker:-



Second request from Producer to Broker



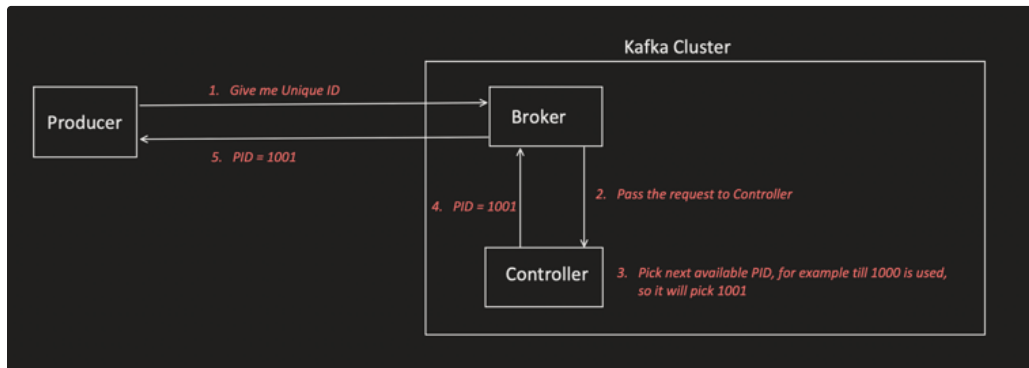
Now within Broker:-



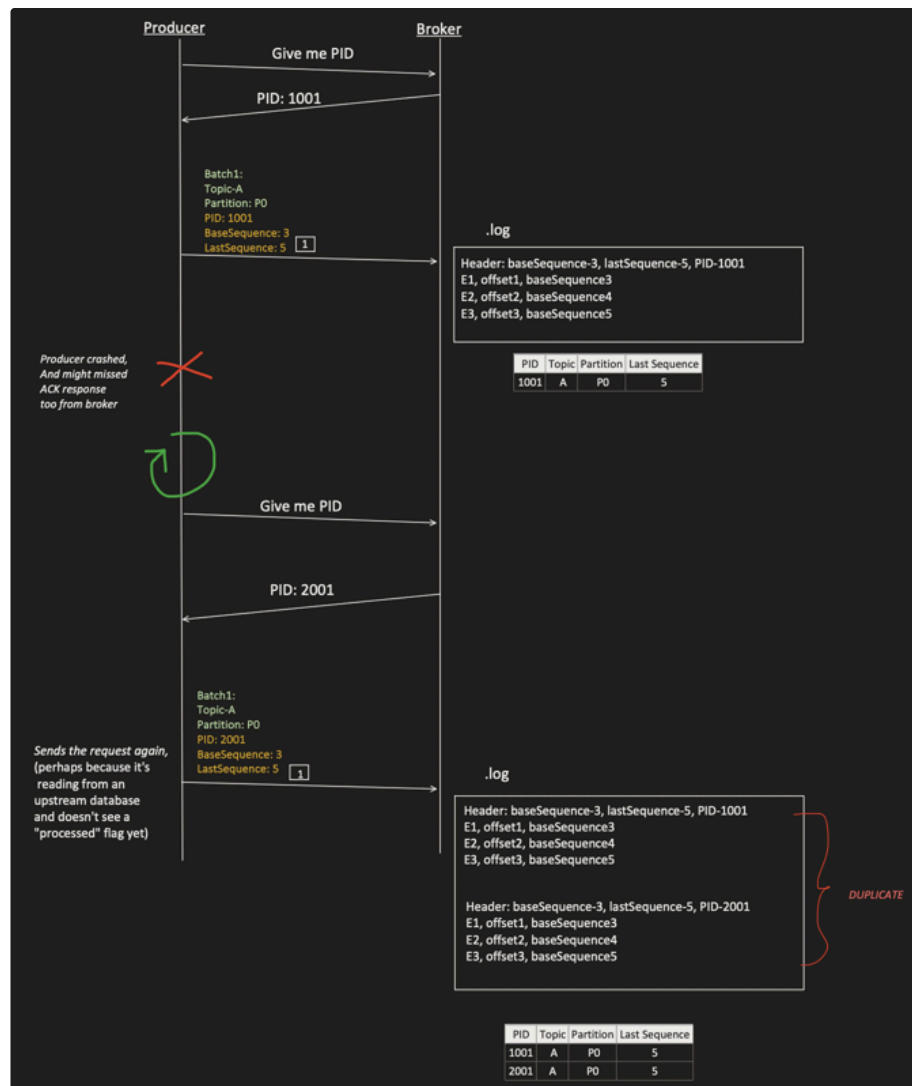
With this both duplicate and message ordering issue is fixed. **But there is 1 loop hole in it.**

If you double check the Step1, you might find 1 loop hole that Controller do not store PID information in its cluster metadata.

Step1: Producer application startup



So, when Producer restart, it will send request again "Give me Unique ID" and since Controller (in case of Idempotency) do not store any producer and PID mapping info, so it will always generate new PID and revert back.



And that's where TRANSACTION comes into the picture, which we will cover after Consumer and Consumer Group Setup.

But 1 advantage of Transaction is that, even when producer restart, Controller will always return same PID. Will see later.

Last but not the least, when we use IDEMPOTENCY:

1. `spring.kafka.producer.acks= all`

Why? Say `ack=1`, leader persisted the event with seq no: 1 but crashed before followers pulls it.

Now let say new leader is elected for specific partition.

Producer will retry and now request will go to different leader now, which does not have the event with seq no 1, so it will also write it.

That's why `acks=all` is recommended .

2. **`spring.kafka.producer.retries = MAX_INT`**

with idempotency, retries are safe. So restriction lifted on the number of retries.

3. **`spring.kafka.producer.properties.max.in.flight.requests.per.connection <= 5`**

Kafka says that idempotency it can only guarantee up to 5 sequence no (Topic, Partition, PID wise).

In other words, Broker maintains a max buffer for only 5 sequence no (for each topic, partition and PID).

So we need to assign the value `<=5`